

Sujet d'examen UE NFE204 - Session 1

Année universitaire 2020-2021

Examen première session : 2 février 2021

Consignes

Durée de l'examen : 3h00 ; Documents autorisés ; Calculatrice autorisée

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 6 pages.

Important : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

Première partie : modélisation NoSQL (8 pts)

Dans le monde d'après, il n'y a plus de presse, de quotidiens ou de magazines. Des journalistes indépendants, payés au caractère, écrivent des articles et les proposent à des plateformes de diffusion. Ces journalistes sont (faiblement) rémunérés en fonction du nombre de personnes qui lisent l'article sur ces plateformes.

Posons-nous quelques questions sur le fonctionnement de cette presse ubérisée. Un modèle UML très sommaire est donné dans la figure 1. Les journalistes publient des articles, et ces articles sont diffusés par des plateformes (plusieurs plateformes si possible, qui elles-mêmes publient beaucoup d'articles).

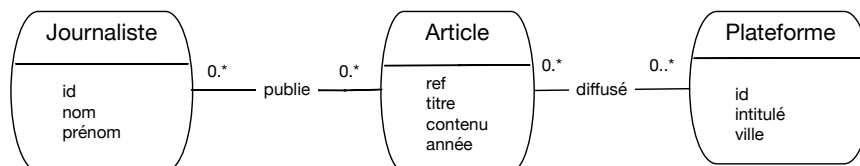


FIGURE 1 – Modèle (sommaire) de la diffusion d'articles ubérisés

Au départ, on représente tout cela avec une base relationnelle dont voici un tout petit échantillon.

ref	titre	année
ju21	Alerte aux gilets jaunes	2018
ju798	Aura-t-on des vaccins pour Noël	2021

La table *Article*

id	nom
nt	N. Travers
pr	P. Rigaux
rfs	R. Fournier-S'niehotta
cdm	C. du Mouza

La table *Journaliste*

refArticle	idAuteur
ju21	cdm
ju21	pr
ju21	nt
ju798	pr

La table des publications

idPlateforme	nom
pj2	Béru
pj5	Croco
pj6	Vautour

La table des plateformes

s refArticle	idPlateforme
ju21	pj2
ju798	pj6
ju21	pj6

La table des diffusions

1. Proposez un format de document JSON représentant toutes les informations présentes dans les tables ci-dessus et relatives à l'auteur *P. Rigaux* (représentation A).

Solution :

```
{ "_id": "pr",
  "nom": "P. Rigaux",
  "articles": [
    { "titre": "Alerte aux gilets jaunes",
      "année": 2018
      "plateformes": ["Béru", "Vautour"]
    },
    { "titre": "Aura-t-on des vaccins pour Noël",
      "année": 2021
      "plateformes": ["Vautour"],
    }
  ]
}
```

2. Proposez un document JSON représentant toutes les informations relatives à la plateforme *Vautour* présentes dans les tables ci-dessus (représentation B).

Solution :

```
{ "_id": "pj6",
  "nom": "Vautour",
  "articles": [
    { "titre": "Alerte aux gilets jaunes",
      "année": 2018
      "auteurs": ["N. Travers", "P. Rigaux", "R. Fournier-S'niehotta"]
    },
    { "titre": "Aura-t-on des vaccins pour Noël",
      "année": 2021
      "auteurs": ["P. Rigaux"],
    }
  ]
}
```

3. Expliquez le calcul MapReduce permettant de produire les documents de type B à partir des documents de type A.

Solution : Voici la modélisation en Javascript. Il fut, dans le Map, deux boucles imbriquées pour accéder aux plateformes dont le nom constitue la clé de regroupement dans la phase de Reduce.

```

function mapAuteurVersPlateforme (doc) {
  // Première boucle sur les articles
  for article in doc.articles {
    // Second boucle sur les plateformes
    for (plateforme in article.plateforme) {
      // La clé est donc le nom de la plateforme. On l'accompagne
      // d'un document contenant l'auteur, le titre et l'année
      emit (plateforme, {"auteur": doc.nom, "titre": article.titre,
                        "année": article.année})
    }
  }
}

```

Pour le Reduce : on reçoit, pour chaque plateforme, la liste des articles avec leurs auteurs. Il suffit de regrouper les articles par titre pour obtenir leurs auteurs.

```

function reduceAuteurVersPlateforme (plateforme, [listeArticles]) {
  // On va s'épargner le petit algo de regroupement des auteurs sur le titre
  les_articles = groupement_auteurs_par_titre (listeArticles)
  // On a tout ce qu'il faut
  return ({ "nom": plateforme, "articles": les_articles })
}

```

Important : pour spécifier les calculs MapReduce, donner les fonctions de Map et de Reduce, en javascript, en pseudo-code, au pire en langage naturel en étant le plus précis possible.

««««< .mine

4. On suppose que le contenu des quatre tables est accessible à un processeur Pig latin, et on veut obtenir à partir de la base de données un document représentant un article. Le document ci-dessous, incomplet, représente un des articles. =====
5. On suppose que le contenu de la base est accessible à un processeur Pig latin sous forme de trois collections. La collection des articles avec des documents du type :

```

{"ref" : "ju21", "titre": "Alerte aux gilets jaunes", "année": 2018}

```

La collection des auteurs. Exemples :

```

{"nom" : "P. Rigaux", "refArticle": "ju21"}
{"nom" : "P. Rigaux", "refArticle": "ju798"}

```

Et enfin la collection des diffusions. Exemples :

```

{"plateforme" : "Béru", "refArticle": "ju21"}
{"plateforme" : "Vautour", "refArticle": "ju798"}

```

On veut obtenir à partir de la base de données un document représentant les informations sur un article »»»»> .r4027

```

{"ref": "ju21",
 "titre" : "Alerte aux gilets jaunes",
 "année": 2018,
 "auteurs": [
   ],
 "plateformes": [
   ]
}

```

- (a) Donnez le contenu JSON des champs auteurs et plateformes d'après la base relationnelle.

- (b) Quelle chaîne de traitement permet de passer des quatre tables indépendantes à la représentation sous forme de document ci-dessus ? Vous pouvez prendre comme hypothèses que les 4 tables existent sous forme de collection dans Pig, et exprimer la chaîne de traitement avec des opérateurs (un schéma commenté fait très bien l'affaire).

Solution : L'opérateur principal à mettre en œuvre est le cogroup. Le premier (mais l'ordre n'est pas important) groupe les articles et les auteurs.

```
articleEtAuteur = cogroup Articles by ref, Auteurs by refArticle;
```

Puis on regroupe la collection obtenue avec la collection des diffusions.

```
articlesComplet = cogroup articleEtAuteur by ref, Diffusions by refArticle;
```

- (c) À votre avis combien faut-il d'opérations MapReduce successives pour effectuer ce calcul ?

Solution : Clairement, il en faut deux.

Deuxième partie : flux (4 pts)

Chaque fois que quelqu'un lit un article sur une plateforme, un message est produit. Le format de ce message est le suivant :

```
{"refArticle": "ju9", "contribution": 0.005}
```

La contribution est le montant (en Euros) généré par cette lecture. Ces messages arrivent en un flux continu. On veut écrire un traitement de flux qui cumule le montant des contributions par article.

1. En vous inspirant des chaînes de traitement sur les flux vues dans le dernier chapitre du cours, indiquez comment on maintient, pour chaque article, un compteur du nombre de lectures.

Solution : Voici la chaîne de traitement en Scala, décalquée à peu de choses près du compteur de mots.

```
case class Lecture(refArticle: String, contribution: float)
val w = lectures.keyBy("refArticle") // Regroupement par article
    .sum("contribution") // Cumul à chaque nouvelle lecture
    .print() // Optionnel: affiche l'évolution continue du cumul
senv.execute("Cumul des contributions")
```

2. On veut rémunérer les auteurs d'articles toutes les 6 heures. Quelle est la technique à utiliser ?

Solution : Il faut évidemment faire un fenêtrage. On maintient un accumulateur accum dans lequel on ajoute les lectures reçues sur le flux, et ce pendant la durée de la fenêtre.

```
case class Lecture(refArticle: String, contribution: float)
val w = lectures.keyBy("refArticle") // Regroupement par article
    .window(6*60*60) // Si on exprime en secondes
    .reduce( (accum, lecture) => (accum.ref, accum + lecture.contribution) )
    .print()
senv.execute("")
```

3. Quelle est la parallélisation maximale permise par ce traitement ?

Solution : Il y a un sous-flux par article, ce qui permet un large parallélisme a priori.

Troisième partie : systèmes distribués (4 pts)

Voici quelques aspects d'Elastic Search qui devraient vous être familiers, et d'autres qui n'ont pas été abordés en cours

La documentation nous dit : *When you index a document, it is stored on a single primary shard determined by a simple formula :*

$$\text{shard} = \text{hash}(\text{id}) \% \text{number_of_primary_shards}$$

Voici la configuration initiale de votre *cluster* ElasticSearch pour les questions qui suivent (rappel : réplica = n signifie $n + 1$ copies d'un document).

- Nombre de fragments (*shards*) : 3
- Nombre de réplicas : 2

Et je crée un index. Chacune des questions ci-dessous vaut 1 point. Une brève explication sera appréciée.

1. Est-ce que mon *cluster* peut être constitué d'un seul nœud (un seul serveur) ?

Solution : Oui, mais l'index sera en situation périlleuse puisqu'il manquera deux serveurs pour héberger les réplicas. Autre réponse : non car il faut stocker les réplicas sur des serveurs distincts.

2. Est-ce que je peux augmenter le nombre de *shards* sans recréer l'index ?

Solution : Non, c'est du hachage statique, toute affectation à un fragment est définitive, sauf à reconstituer l'index.

3. Est-ce que je peux augmenter le nombre de réplicas ?

Solution : Oui, aucun problème, à condition d'avoir assez de serveurs sinon ça n'a pas vraiment de sens.

4. Est-ce que je peux ajouter des nœuds à mon *cluster* ?

Solution : Oui, ça peut améliorer la distribution des fragments.

5. Si oui, à partir de quand est-ce que ça ne sert plus à rien d'ajouter des nœuds, en prenant en compte la configuration initiale ?

Solution : Quand on a un fragment par serveur, ça ne sert à rien d'en ajouter. Donc $3 * (2 + 1) = 9$ serveurs.

Un dernier point à gagner. Un de vos collègues vous affirme qu'il vaut mieux créer un index avec beaucoup de *shards*, pour anticiper la croissance future. Voici un argument contre ce choix, issu de la documentation. *Term statistics, used to calculate relevance, are per shard. Having a small amount of data in many shards leads to poor relevance.*

Quatrième partie : questions de cours (4 pts)

Nous avons vu que dans un système comme cassandra, on pouvait régler des paramètres W (acquittements en écriture) et R (acquittements en lecture).

Ce réglage correspond à la recherche d'un compromis, mais entre quelles propriétés ? Reprenez le théorème CAP et donnez des réponses argumentées aux questions suivantes :

1. Si W est fixé à 1, quelle(s) propriété(s) est/sont sacrifiée(s) avec une faible valeur de R Et avec la valeur de R maximale ?

Solution : Si R est faible, on sacrifie la cohérence car on augmente la probabilité de lire une valeur obsolète. En augmentant R on améliore la cohérence, mais on diminue la disponibilité et la tolérance.

2. Si W est fixé à la valeur maximale, le choix de R influe-t-il sur les propriétés CAP ?

Solution : Non, si on a réussi à faire l'écriture avec tous les acquittements, toute lecture ramène une valeur cohérente, la disponibilité et la tolérance sont aussi maximales.